Practical:10	Implement Kruskal's algorithm.
__/__ /2026	

Objective

- To implement Kruskal's algorithm to find the Minimum Spanning Tree (MST) of a connected, weighted, undirected graph.
- To understand how the greedy method picks edges in non-decreasing order of weight using Disjoint Set Union (Union-Find).
- To analyze the time and space complexity of Kruskal's algorithm.

Program

```
# Kruskal's Minimum Spanning Tree Algorithm using Disjoint Set Union (DSU)


class DisjointSet:
    def __init__(self, n):
        self.parent = list(range(n))
        self.rank = [0] * n

    def find(self, i):
        # Path compression
        if self.parent[i] == i:
            return i
        self.parent[i] = self.find(self.parent[i])
        return self.parent[i]

    def union(self, i, j):
        # Union by rank
        root_i = self.find(i)
        root_j = self.find(j)
        if root_i != root_j:
            if self.rank[root_i] < self.rank[root_j]:
                root_i, root_j = root_j, root_i
            self.parent[root_j] = root_i
```

```
            if self.rank[root_i] == self.rank[root_j]:
                self.rank[root_i] += 1
            return True
        return False

# Graph represented as an edge list: (u, v, weight)
edges = [
    (0, 1, 4),
    (0, 2, 3),
    (1, 2, 1),
    (1, 3, 2),
    (2, 3, 4),
    (3, 4, 2),
    (3, 5, 7),
    (4, 5, 6)
]

num_vertices = 6

# Step 1: Sort all edges in non-decreasing order of their weight
sorted_edges = sorted(edges, key=lambda edge: edge[2])

dsu = DisjointSet(num_vertices)
mst_edges = []
total_cost = 0

print("KRUSKAL'S ALGORITHM")
step = 1

# Step 2: Iterate through sorted edges and apply Union-Find
```

```
for u, v, w in sorted_edges:
    if dsu.union(u, v):
        mst_edges.append((u, v, w))
        total_cost += w
        print(f"Step {step}: add edge {u}-{v} (weight {w})")
    else:
        print(f"Step {step}: reject edge {u}-{v} (weight {w}) - creates cycle")
    step += 1

print("\nEdges in the MST:")
for u, v, w in mst_edges:
    print(f"{u}-{v} : {w}")

print("\nTotal cost of MST =", total_cost)
```

Output

```
KRUSKAL'S ALGORITHM
Step 1: add edge 1-2 (weight 1)
Step 2: add edge 1-3 (weight 2)
Step 3: add edge 3-4 (weight 2)
Step 4: add edge 0-2 (weight 3)
Step 5: reject edge 0-1 (weight 4) - creates cycle
Step 6: reject edge 2-3 (weight 4) - creates cycle
Step 7: add edge 4-5 (weight 6)
Step 8: reject edge 3-5 (weight 7) - creates cycle

Edges in the MST:
1-2 : 1
1-3 : 2
3-4 : 2
0-2 : 3
4-5 : 6

Total cost of MST = 14
```

Analysis

a) Problem Analysis

A spanning tree of a connected, undirected graph with V vertices connects all vertices using exactly $V - 1$ edges without forming any cycles. A **Minimum Spanning Tree (MST)** is a spanning tree whose sum of edge weights is minimized. Typical applications include network design, electrical grid wiring, computer networking, and transportation routing at minimum cost.

b) Greedy Approach

Kruskal's algorithm forms the MST by building a forest of trees that gradually merges into a single tree. It processes edges globally in increasing order of weight.

- For each edge (u, v) with weight w, it checks whether vertices u and v belong to the same connected component.
- If they belong to different components, adding edge (u, v) will not form a cycle. The edge is included, and the two components are merged using the **Disjoint Set Union (DSU)** data structure.
- If they belong to the same component, adding the edge creates a cycle, so the edge is discarded.

c) Example Analysis

Consider a graph with 6 vertices (0 to 5) and 8 edges:

Edge	Weight
0–1	4
0–2	3
1–2	1
1–3	2
2–3	4
3–4	2
3–5	7
4–5	6

Trace of Kruskal's Algorithm:

Step	Edge Processed	Weight	Action	Component Structure / Disjoint Sets	Total MST Cost
1	1–2	1	**Included**	$\{0\},\{1,2\},\{3\},\{4\},\{5\}$	1
2	1–3	2	**Included**	$\{0\},\{1,2,3\},\{4\},\{5\}$	3
3	3–4	2	**Included**	$\{0\},\{1,2,3,4\},\{5\}$	5
4	0–2	3	**Included**	$\{0,1,2,3,4\},\{5\}$	8
5	0–1	4	**Rejected** (Cycle)	$\{0,1,2,3,4\},\{5\}$	8
6	2–3	4	**Rejected** (Cycle)	$\{0,1,2,3,4\},\{5\}$	8
7	4–5	6	**Included**	$\{0,1,2,3,4,5\}$	14
8	3–5	7	**Rejected** (Cycle)	$\{0,1,2,3,4,5\}$	14

The resulting MST contains $V - 1 = 5$ edges $(1\text{--}2,1\text{--}3,3\text{--}4,0\text{--}2,4\text{--}5)$ with a minimum cost of **14**.

d) Algorithm Analysis

1. **Sort Edges:** Sort all E edges of the graph in non-decreasing order of weight.
2. **Initialize Disjoint Sets:** Create V disjoint sets, placing each vertex in its own independent set.
3. **Select Edges:** Iterate over the sorted edges. For edge (u, v):
    - Execute find(u) and find(v) to determine their representative roots.
4. **Cycle Prevention:**
    - If find(u) != find(v), add (u, v) to the MST and merge the sets using union(u, v).
    - Otherwise, skip the edge as it forms a cycle.
5. **Termination:** Stop when $V - 1$ edges are added or all edges have been evaluated.

e) Time Complexity

- **Edge Sorting:** Sorting E edges takes $O(E \log E)$ time.
- **Disjoint Set Operations:** Each find and union operation takes $O(\alpha(V))$ amortized time using path compression and union by rank (where α is the inverse Ackermann function, practically $O(1)$).
- **Overall Time Complexity:** $O(E \log E)$ or equivalently $O(E \log V)$ (since $E \leq V^2$, $\log E = O(\log V)$).

f) Space Complexity

- **Edge List Storage:** Storing E edges requires $O(E)$ space.
- **DSU Data Structure:** Array structures for parent and rank require $O(V)$ auxiliary space.
- **Overall Space Complexity:** $O(V + E)$.

Conclusion

Kruskal's algorithm was successfully implemented in Python using the greedy method alongside the Disjoint Set Union (DSU) data structure. For the given graph, it selected 5 edges with a total MST cost of 14. The edge-sorting combined with Union-Find operations results in an efficient overall time complexity of $O(E \log V)$ and space complexity of $O(V + E)$.